

Experiment 3 – Detailed Examples of All 30 Linux Commands

1. man – manual/help

Shows full documentation of a command.

man ls

👉 Opens help page for ls. Use q to exit.

2. ls – list files

ls -l

👉 Displays files with permissions, owner, size, date.

3. echo – print / write text

echo "Hello World"

echo "Hello" > file.txt

👉 Prints text or writes into file.

4. read – take input

read name

echo \$name

👉 Stores user input in variable.

5. cat – display file

cat file.txt

👉 Shows contents of file.

6. cd – change directory

cd /home/user/Documents

👉 Moves into specified directory.

7. mkdir – create directory

mkdir myfolder

👉 Creates a new folder.

8. pwd – print working directory

pwd

👉 Shows current directory path.

9. mv – move/rename

mv file.txt newfile.txt

mv file.txt /home/user/

👉 Renames or moves file.

10. cp – copy file

cp file.txt backup.txt

👉 Creates a copy.

 11. rm – delete file

rm file.txt

rm -r folder

👉 Deletes file or directory.

 12. touch – create file

touch new.txt

👉 Creates empty file.

 13. chmod – change permission

chmod 755 file.sh

👉 Gives execute permission.

 14. chown – change owner

chown user file.txt

👉 Changes file ownership.

 15. more – view file

more file.txt

👉 Shows file page by page.

 16. less – advanced viewer

less file.txt

👉 Scroll forward/backward.

 17. find – search files

find . -name "file.txt"

👉 Finds file in directory.

 18. wc – count

wc file.txt

👉 Shows lines, words, characters.

 19. cut – extract data

cut -d ":" -f1 /etc/passwd

👉 Extracts first column.

 20. paste – merge files

paste file1.txt file2.txt

👉 Combines files line by line.

 21. head – first lines

head -n 5 file.txt

👉 Shows first 5 lines.



22. tail – last lines

```
tail -n 5 file.txt
```

👉 Shows last 5 lines.



23. useradd – add user

```
sudo useradd user1
```

👉 Creates new user.



24. userdel – delete user

```
sudo userdel user1
```

👉 Deletes user.



25. usermod – modify user

```
sudo usermod -aG sudo user1
```

👉 Adds user to group.



26. passwd – change password

```
passwd user1
```

👉 Sets or changes password.



27. top – system monitor

```
top
```

👉 Shows running processes live.



28. ps – process status

```
ps aux
```

👉 Lists all processes.



29. ssh – remote login

```
ssh user@192.168.1.100
```

👉 Connects to remote system.



30. scp – copy remotely

```
scp file.txt user@192.168.1.100:/home/user
```

👉 Transfers file over network.



Experiment 4: Largest Among 3 Numbers



Aim

To write a **shell script** to find the **largest among three numbers**.



Theory

This program uses:

- User input (**read**)
 - Conditional statements (**if-elif-else**)
 - Comparison operators (**-gt**)
-



Algorithm (Step-by-Step)

1. Start
 2. Declare three variables **a**, **b**, and **c**
 3. Read three numbers from the user
 4. Compare **a** with **b** and **c**
 - If **a > b** AND **a > c**, then **a** is largest
 5. Else compare **b** with **c**
 - If **b > c**, then **b** is largest
 6. Else
 - **c** is largest
 7. Display the largest number
 8. Stop
-



Source Code

```
#!/bin/bash
```

```
read -p "Enter 3 numbers: " a b c
```

```
if [ $a -gt $b ] && [ $a -gt $c ]
```

```
then
```

```
    echo "$a is greatest"
```

```
elif [ $b -gt $c ]
```

```
then
```

```
    echo "$b is greatest"
```

```
else
```

```
    echo "$c is greatest"
```

```
fi
```

Example Run

Enter 3 numbers: 10 25 15
25 is greatest

Explanation

- `read` → takes input
 - `-gt` → greater than comparison
 - `if-elif-else` → decision making
-

Result

Successfully created and executed a shell script to find the **largest among three numbers**.

Experiment 8: Even or Odd

Aim

To write a **shell script** to check whether a number is **even or odd**.

Theory

A number is:

- **Even** → if divisible by 2
- **Odd** → if not divisible by 2

This is checked using the **modulus operator (%)** in shell scripting.

Algorithm (Step-by-Step)

1. Start

2. Declare a variable `n`
 3. Read a number from the user
 4. Compute `n % 2`
 5. If result equals 0
 - Print "Even"
 6. Else
 - Print "Odd"
 7. Stop
-



Source Code (Google Docs Ready)

```
#!/bin/bash
```

```
read -p "Enter a number: " n
```

```
if [ $((n % 2)) -eq 0 ]
```

```
then
```

```
    echo "$n is Even"
```

```
else
```

```
    echo "$n is Odd"
```

```
fi
```



Example Run

```
Enter a number: 7
```

```
7 is Odd
```

```
Enter a number: 10
```

```
10 is Even
```



Explanation

- `read` → takes input
- `$(( ))` → arithmetic operation
- `%` → modulus (remainder)
- `-eq` → equal comparison
- `if-else` → decision making

Result

Successfully created and executed a shell script to determine whether a number is **even** or **odd**.

Experiment 11: Reverse a Number

Aim

To write a **shell script** to reverse a given number.

Theory

Reversing a number involves:

- Extracting digits using **modulus (%)**
 - Removing digits using **division (/)**
 - Building the reversed number step by step
-

Algorithm (Step-by-Step)

1. Start
 2. Declare variables `n`, `rev = 0`, `rem`
 3. Read a number from the user
 4. Repeat while `n > 0`
 - Find remainder: `rem = n % 10`
 - Update reverse: `rev = rev * 10 + rem`
 - Remove last digit: `n = n / 10`
 5. Display reversed number
 6. Stop
-

Source Code (Google Docs Ready)

```
#!/bin/bash
```

```
read -p "Enter a number: " n
rev=0

while [ $n -gt 0 ]
do
    rem=$((n % 10))
    rev=$((rev * 10 + rem))
    n=$((n / 10))
done

echo "Reversed number is: $rev"
```

Example Run

Enter a number: 1234
Reversed number is: 4321

Explanation

- `% 10` → extracts last digit
 - `/ 10` → removes last digit
 - Loop continues until number becomes 0
 - Builds reversed number step by step
-

Result

Successfully created and executed a shell script to **reverse a number**.

Experiment 15: Factorial

Aim

To write a **shell script** to find the **factorial of a number**.

Theory

Factorial of a number n is:

👉 $n! = n \times (n-1) \times (n-2) \times \dots \times 1$

Example:

👉 $5! = 5 \times 4 \times 3 \times 2 \times 1 = 120$

Algorithm (Step-by-Step)

1. Start
 2. Declare variables n , $fact = 1$, i
 3. Read a number from the user
 4. Initialize $i = 1$
 5. Repeat while $i \leq n$
 - $fact = fact * i$
 - Increment i by 1
 6. Display factorial
 7. Stop
-

Source Code (Google Docs Ready)

```
#!/bin/bash

read -p "Enter a number: " n
fact=1
i=1

while [ $i -le $n ]
do
    fact=$((fact * i))
    i=$((i + 1))
done

echo "Factorial of $n is: $fact"
```

Example Run

Enter a number: 5

Factorial of 5 is: 120

Explanation

- Loop multiplies numbers from 1 to `n`
 - `fact` stores result
 - `$(( ))` is used for arithmetic
-

Result

Successfully created and executed a shell script to find the **factorial of a number**.

Experiment 16: Palindrome

Aim

To write a **shell script** to check whether a number is a **palindrome**.

Theory

A number is a **palindrome** if it remains the same when reversed.

Examples:

- 121 → Palindrome
- 123 → Not Palindrome

The logic is:

- Reverse the number
 - Compare with original number
-

Algorithm (Step-by-Step)

1. Start
 2. Declare variables `n`, `temp`, `rev = 0`, `rem`
 3. Read a number from the user
 4. Store original number in `temp`
 5. Repeat while `n > 0`
 - `rem = n % 10`
 - `rev = rev * 10 + rem`
 - `n = n / 10`
 6. Compare `rev` with `temp`
 - If equal → Palindrome
 - Else → Not Palindrome
 7. Display result
 8. Stop
-



Source Code (Google Docs Ready)

```
#!/bin/bash
```

```
read -p "Enter a number: " n
temp=$n
rev=0
```

```
while [ $n -gt 0 ]
do
```

```
    rem=$((n % 10))
    rev=$((rev * 10 + rem))
    n=$((n / 10))
done
```

```
if [ $rev -eq $temp ]
then
    echo "$temp is a Palindrome"
else
    echo "$temp is not a Palindrome"
fi
```



Example Run

Enter a number: 121

121 is a Palindrome

Enter a number: 123

123 is not a Palindrome

Explanation

- Reverse logic same as Experiment 11
 - Compare reversed number with original
 - If equal → palindrome
-

Result

Successfully created and executed a shell script to check whether a number is a **palindrome**.

Experiment 17: Fibonacci Series

Aim

To write a **shell script** to generate the **Fibonacci series** up to **n** terms.

Theory

Fibonacci series is a sequence where:

👉 Each number = sum of previous two numbers

Example:

0 1 1 2 3 5 8 ...

Algorithm (Step-by-Step)

1. Start
2. Declare variables **n**, **a = 0**, **b = 1**, **c**, **i = 1**

3. Read number of terms *n*
 4. Print first two terms (*a* and *b*)
 5. Repeat while *i* <= *n*-2
 - *c* = *a* + *b*
 - Print *c*
 - *a* = *b*
 - *b* = *c*
 - Increment *i*
 6. Stop
-



Source Code (Google Docs Ready)

```
#!/bin/bash

read -p "Enter number of terms: " n

a=0
b=1

echo "Fibonacci Series:"

echo -n "$a $b "

i=1
while [ $i -le $((n-2)) ]
do
    c=$((a + b))
    echo -n "$c "
    a=$b
    b=$c
    i=$((i + 1))
done

echo
```



Example Run

Enter number of terms: 6
Fibonacci Series:

0 1 1 2 3 5

Explanation

- a and $b \rightarrow$ first two numbers
 - Loop generates next terms
 - $c = a + b \rightarrow$ next value
 - Values are updated each iteration
-

Result

Successfully created and executed a shell script to generate the **Fibonacci series**.

Experiment 21: GCD and LCM

Aim

To write a **shell script** to find the **GCD (Greatest Common Divisor)** and **LCM (Least Common Multiple)** of two numbers.

Theory

- **GCD**: Largest number that divides both numbers
- **LCM**: Smallest number divisible by both

 Formula:
 $LCM = (a \times b) / GCD$

Algorithm (Step-by-Step)

♦ For GCD (Euclidean Method)

1. Start
2. Read two numbers a and b

3. Store original values
4. While $b \neq 0$
 - $temp = b$
 - $b = a \% b$
 - $a = temp$
5. $GCD = a$

◆ For LCM

6. $LCM = (original_a \times original_b) / GCD$
7. Display GCD and LCM
8. Stop



Source Code (Google Docs Ready)

```
#!/bin/bash
```

```
read -p "Enter two numbers: " a b
```

```
x=$a
```

```
y=$b
```

```
# GCD calculation
```

```
while [ $b -ne 0 ]
```

```
do
```

```
    temp=$b
```

```
    b=$((a % b))
```

```
    a=$temp
```

```
done
```

```
gcd=$a
```

```
# LCM calculation
```

```
lcm=$((x * y / gcd))
```

```
echo "GCD = $gcd"
```

```
echo "LCM = $lcm"
```



Example Run

Enter two numbers: 12 18

GCD = 6

LCM = 36

Explanation

- `%` → remainder (used in GCD)
 - Loop reduces values step by step
 - LCM uses formula with GCD
-

Result

Successfully created and executed a shell script to find **GCD and LCM of two numbers**.

Experiment 12: Combine Strings

Aim

To write a **shell script** to **combine (concatenate) two strings**.

Theory

- String concatenation means **joining two or more strings**
 - In shell scripting, strings are combined simply by placing them together
-

Algorithm (Step-by-Step)

1. Start
2. Declare two variables `str1` and `str2`
3. Read two strings from the user
4. Combine the strings
5. Store result in a variable
6. Display the combined string

7. Stop



Source Code (Google Docs Ready)

```
#!/bin/bash
```

```
read -p "Enter first string: " str1  
read -p "Enter second string: " str2
```

```
result="$str1$str2"
```

```
echo "Combined string: $result"
```



Example Run

```
Enter first string: Hello  
Enter second string: World  
Combined string: HelloWorld
```



Explanation

- Strings are combined using "\$str1\$str2"
 - No special operator needed
 - Variables store string values
-



Result

Successfully created and executed a shell script to **combine two strings**.



Experiment 13: Substring



Aim

To write a **shell script** to extract a **substring** from a given string.

Theory

- A **substring** is a part of a string
 - In shell scripting, substring can be extracted using:
👉 `${string:start:length}`
-

Algorithm (Step-by-Step)

1. Start
 2. Declare variables `str`, `start`, `len`
 3. Read a string from the user
 4. Read starting position
 5. Read length of substring
 6. Extract substring using `${str:start:length}`
 7. Display the result
 8. Stop
-

Source Code (Google Docs Ready)

```
#!/bin/bash

read -p "Enter a string: " str
read -p "Enter starting position: " start
read -p "Enter length: " len

sub=${str:$start:$len}

echo "Substring: $sub"
```

Example Run

```
Enter a string: HelloWorld
Enter starting position: 0
Enter length: 5
Substring: Hello
```

Explanation

- `${str:start:length}` → extracts substring
 - Index starts from **0**
 - `start` → position
 - `length` → number of characters
-

Result

Successfully created and executed a shell script to extract a **substring from a string**.

Experiment 18: Reverse a String

Aim

To write a **shell script** to **reverse a given string**.

Theory

Reversing a string means printing characters in **reverse order**.

Example:

👉 `Hello` → `olleH`

This can be done by:

- Iterating from last character to first
 - Building a new reversed string
-

Algorithm (Step-by-Step)

1. Start
2. Declare variables `str`, `rev=""`, `len`, `i`

3. Read a string from the user
 4. Find length of string
 5. Set `i = len - 1`
 6. Repeat while `i >= 0`
 - Extract character at position `i`
 - Append to `rev`
 - Decrement `i`
 7. Display reversed string
 8. Stop
-



Source Code (Google Docs Ready)

```
#!/bin/bash
```

```
read -p "Enter a string: " str
rev=""
```

```
len=${#str}
```

```
for (( i=len-1; i>=0; i-- ))
do
    rev="$rev${str:$i:1}"
done
```

```
echo "Reversed string: $rev"
```



Example Run

```
Enter a string: Hello
Reversed string: olleH
```



Explanation

- `${#str}` → gives length
- `${str:$i:1}` → extracts one character
- Loop runs from end to start
- Builds reversed string step by step

Result

Successfully created and executed a shell script to **reverse a string**.

Experiment 6: Number of Terminals Logged

Aim

To write a **shell script** to find the **number of terminals a user is logged into**.

Theory

- The **who** command shows **currently logged-in users**
 - Using **grep**, we can filter a specific user
 - Using **-c**, we can **count occurrences (terminals)**
-

Algorithm (Step-by-Step)

1. Start
 2. Check if username is provided as argument
 3. If not, display message to enter username
 4. Use **who** command to list logged-in users
 5. Store output in a file
 6. Use **grep -c** to count occurrences of username
 7. Display number of terminals
 8. Stop
-

Source Code (Google Docs Ready)

```
#!/bin/bash
```

```
if [ $# -eq 1 ]
```

```
then
```

```
    who > users.txt
```

```
    echo "$1 user is logged in:"
    grep -c $1 users.txt
else
    echo "Please enter username"
fi
```

Example Run

```
./script.sh user1
user1 user is logged in:
2
```

Explanation

- `who` → shows logged-in users
 - `grep -c` → counts how many times user appears
 - `$1` → command line argument
 - `$#` → number of arguments
-

Result

Successfully created and executed a shell script to find the **number of terminals a user is logged into**.

Experiment 14: System Config

Aim

To write a **shell script** to display **system configuration details**.

Theory

- Environment variables store system information
- Commands like `pwd` give runtime details

- Useful for checking system setup
-



Algorithm (Step-by-Step)

1. Start
 2. Display current shell using `$SHELL`
 3. Display home directory using `$HOME`
 4. Display OS type using `$OSTYPE`
 5. Display current working directory using `pwd`
 6. Stop
-



Source Code (Your Version – Google Docs Ready)

```
#!/bin/bash
```

```
echo "Current Shell: $SHELL"
echo "Home Directory: $HOME"
echo "Operating System Type: $OSTYPE"
echo "Current working directory: "
pwd
```



Example Run

```
Current Shell: /bin/bash
Home Directory: /home/akhil
Operating System Type: linux-gnu
Current working directory:
/home/akhil/Documents
```



Explanation

- `$SHELL` → shows current shell
- `$HOME` → user home directory

- `$OSTYPE` → OS type
 - `pwd` → current working directory
-

Result

Successfully created and executed a shell script to display **system configuration details**.

Experiment 20: System Information

Aim

To write a **shell script** for:

- i) Showing the **count of users logged in**
 - ii) Printing **column list of files in home directory**
 - iii) Listing **jobs with below normal priority**
-

Theory

- `who` → shows logged-in users
 - `wc -l` → counts users
 - `ls -l + cut` → displays selected file columns
 - `ps -al` → shows process details
-

Algorithm (Step-by-Step)

1. Start
 2. Use `who` and count users using `wc -l`
 3. Display selected columns of files using `ls -l` and `cut`
 4. Use `ps -al` to list processes
 5. Extract required columns
 6. Display results
 7. Stop
-



Source Code (Google Docs Ready)

```
#!/bin/bash
```

```
echo "1. Count of users logged in:"  
who | wc -l
```

```
echo "-----"
```

```
echo "2. Column list of files in home directory:"  
ls -l $HOME | cut -c 17-24,39-50,56-
```

```
echo "-----"
```

```
echo "3. List of below priority jobs:"  
ps -al | cut -c 16-19,70-
```



Example Output

```
1. Count of users logged in:  
3
```

```
-----
```

```
2. Column list of files in home directory:  
4096 Jan 12 file1.txt  
2048 Feb 10 script.sh
```

```
-----
```

```
3. List of below priority jobs:  
1234 firefox  
2345 chrome
```



Explanation

- `who | wc -l` → counts logged-in users
- `ls -l $HOME | cut` → extracts file size, date, name
- `ps -al | cut` → extracts process details

Result

Successfully created and executed a shell script to display **user count, file details, and below priority jobs**.

Experiment 22: Student Database

Aim

Shell script to perform database operations for student data like **view, add and delete records**.

Theory

- Data is stored in a **text file (database)**
 - Each record is stored in a single line
 - Commands used:
 - **cat** → view records
 - **grep** → search/delete
 - **>>** → add records
 - **case** → menu-driven control
-

Algorithm (Step-by-Step)

1. Start
 2. Read database name
 3. Display menu
 4. Read user choice
 5. Perform operation (view/search/add/delete)
 6. Ask to continue
 7. Repeat until exit
 8. Stop
-

Source Code (Simple & Exam Ready)

```
#!/bin/bash

ch_cont="y"
read -p "Enter database name: " db

while [ "$ch_cont" = "y" ]
do
    echo "1. View Database"
    echo "2. View Specific Record"
    echo "3. Add Record"
    echo "4. Delete Record"
    echo "5. Exit"

    read -p "Enter choice: " ch

    case $ch in
        1) cat "$db" ;;

        2) read -p "Enter ID: " id
            grep "^$id " "$db" ;;

        3) read -p "Enter ID Name Course College: " rec
            echo "$rec" >> "$db" ;;

        4) read -p "Enter ID: " id
            grep -v "^$id " "$db" > temp
            mv temp "$db"
            echo "Record deleted" ;;

        5) exit ;;

        *) echo "Invalid choice" ;;

    esac

    read -p "Continue? (y/n): " ch_cont
done
```



Example Run

1. View Database

2. View Specific Record
3. Add Record
4. Delete Record
5. Exit

Enter choice: 3

Enter ID Name Course College: 101 Akhil MCA CET

Explanation

- `cat` → display records
 - `grep "^id"` → exact match
 - `grep -v` → delete record
 - `>>` → add record
 - `while` → loop execution
-

Result

Successfully created and executed a shell script to perform **student database operations**.

Experiment 5: Simple Calculator

Aim

To write a **shell script** to perform **basic arithmetic operations**.

Theory

- Uses `read` for input
 - Uses `$(( ))` for arithmetic
 - Uses `case` for selecting operation
-



Algorithm (Step-by-Step)

1. Start
 2. Read two numbers
 3. Display menu (Add, Subtract, Multiply, Divide)
 4. Read user choice
 5. Perform operation using **case**
 6. Display result
 7. Stop
-



Source Code (Exam Ready)

```
#!/bin/bash
```

```
read -p "Enter two numbers: " a b
```

```
echo "1. Addition"
```

```
echo "2. Subtraction"
```

```
echo "3. Multiplication"
```

```
echo "4. Division"
```

```
read -p "Enter choice: " ch
```

```
case $ch in
```

```
1) echo "Result = $((a + b))" ;;
```

```
2) echo "Result = $((a - b))" ;;
```

```
3) echo "Result = $((a * b))" ;;
```

```
4) echo "Result = $((a / b))" ;;
```

```
*) echo "Invalid choice" ;;
```

```
esac
```



Example Run

```
Enter two numbers: 8 2
```

```
Enter choice: 3
```

```
Result = 16
```

Explanation

- `read` → takes input
 - `$(( ))` → performs arithmetic
 - `case` → selects operation
-

Result

Successfully created and executed a shell script to perform **basic arithmetic operations**.

Experiment 7: Reverse Content of File

Aim

To write a **shell script** to reverse the contents of a file.

Source Code (Simple Version)

```
#!/bin/bash

read -p "Enter file name: " file

if [ -f "$file" ]
then
    echo "Original content:"
    cat "$file"

    echo "Reversed content:"
    tac "$file"
else
    echo "File does not exist"
fi
```

Example Output

Enter file name: data.txt

Original content:

Line1

Line2

Line3

Reversed content:

Line3

Line2

Line1

Result

Successfully reversed the **contents of a file**.

Experiment 9: Check Directory

Aim

To write a **shell script** to check whether a given input is a **directory or not**.

Theory

- Directories are special files in Linux
 - `-d` is used to check if a directory exists
-

Algorithm (Step-by-Step)

1. Start
 2. Read directory name from user
 3. Check using `-d`
 4. If true → print "Directory exists"
 5. Else → print "Not a directory"
 6. Stop
-



Source Code (Your Style – with loop)

```
#!/bin/bash
```

```
read -p "Enter directory name you want to search: " dir
```

```
for filename in E:/MCA/S2/lab/$dir
do
    if [ -d "$filename" ]
    then
        echo "$filename is a directory"
    else
        echo "$filename is not a directory"
    fi
done
```



Example Run

```
Enter directory name you want to search: test
E:/MCA/S2/lab/test is a directory
```



Explanation

- `for filename in path` → assigns path to variable
 - `-d` → checks directory existence
 - `$dir` → user input
 - `if-else` → decision making
-



Result

Successfully created and executed a shell script to **check whether a given input is a directory or not.**



Experiment 10: Count Number of Files



Aim

To write a **shell script** to count the **number of files in a directory**.

Source Code (Your Style – with fixed path)

```
#!/bin/bash
```

```
read -p "Enter directory name you want to search: " dir
```

```
direct="E:/MCA/S2/lab/$dir"
```

```
if [ -d "$direct" ]
```

```
then
```

```
    count=$(ls "$direct" | wc -l)
```

```
    echo "Number of files in $direct = $count"
```

```
else
```

```
    echo "Directory not found"
```

```
fi
```

Example Run

```
Enter directory name you want to search: test
```

```
Number of files in E:/MCA/S2/lab/test = 6
```

Explanation

- `direct="E:/MCA/S2/lab/$dir"` → creates full path
 - `-d` → checks if directory exists
 - `ls` → lists files
 - `wc -l` → counts number of files
-

Note (for viva)

- Counts **files + folders**
- Does **not include hidden files**

Result

Successfully created and executed a shell script to **count the number of files in a directory**.

Experiment 19: File Handling – 1

Aim

To write shell scripts to perform basic file handling operations:

- Create & copy file
- Print from a certain line
- Find difference between files
- Pattern matching

Source Code (All in One – Exam Ready)

```
#!/bin/bash
```

```
echo "1. Create & Copy File"
echo "2. Print from Line"
echo "3. Difference Between Files"
echo "4. Pattern Matching"
echo "5. Exit"
```

```
read -p "Enter choice: " ch
```

```
case $ch in
```

```
1)
```

```
    read -p "Enter file1: " f1
    echo "Enter content:"
    cat > "$f1"
```

```
    read -p "Enter file2: " f2
    cp "$f1" "$f2"
    cat "$f2"
```

```
;;
```

2)

```
read -p "Enter file name: " f
echo "Enter content:"
cat > "$f"
```

```
read -p "Enter line number: " l
tail -n +"$l" "$f"
;;
```

3)

```
read -p "Enter file1: " f1
read -p "Enter file2: " f2
```

```
echo "Enter content for $f1:"
cat > "$f1"
```

```
echo "Enter content for $f2:"
cat > "$f2"
```

```
diff "$f1" "$f2"
;;
```

4)

```
read -p "Enter file name: " f
echo "Enter content:"
cat > "$f"
```

```
read -p "Enter pattern: " p
grep -ni "$p" "$f"
;;
```

5) exit ;;

*) echo "Invalid choice" ;;

esac

Example Run

Enter choice: 1

Enter file1: a.txt

Enter content:

Hello

Ctrl+D

Enter file2: b.txt

Hello

Explanation

- `cat > file` → create file
- `cp` → copy file
- `tail -n +n` → print from line
- `diff` → compare files
- `grep -ni` → pattern search

Result

Successfully created and executed a shell script to perform **file handling operations (create, copy, display, compare, search)**.

Experiment 23: File Handling – 2

Aim

To perform file operations using Linux commands:

1. Create a file using `cat`
2. Display first and last lines
3. Copy file
4. Count lines, words, characters
5. Display a specific line
6. Append contents to another file

Algorithm (Step-by-Step)

1. Create file `myfile` using `cat`

2. Use **head** to print first 2 rows
 3. Use **tail** to print last 2 rows
 4. Copy file using **cp**
 5. Count using **wc**
 6. Print 3rd row using **sed**
 7. Append using **>>**
-

Source Code (Complete – Exam Ready)

```
#!/bin/bash
```

```
# 1. Create file
```

```
cat > myfile <<EOF
```

```
1001 RAM 97
```

```
1002 NEHA 89
```

```
1010 DIVYA 77
```

```
1025 RAHUL 65
```

```
1012 ARUN 99
```

```
1013 NISHA 76
```

```
EOF
```

```
echo "-----"
```

```
# 2. First 2 rows
```

```
echo "First 2 rows:"
```

```
head -2 myfile
```

```
echo "-----"
```

```
# 3. Last 2 rows
```

```
echo "Last 2 rows:"
```

```
tail -2 myfile
```

```
echo "-----"
```

```
# 4. Copy file
```

```
cp myfile mycopyfile
```

```
echo "File copied to mycopyfile"
```

```
echo "-----"
```

```
# 5. Count
echo "Lines: $(wc -l < myfile)"
echo "Words: $(wc -w < myfile)"
echo "Characters: $(wc -c < myfile)"
```

```
echo "-----"
```

```
# 6. 3rd row
echo "3rd row:"
sed -n '3p' myfile
```

```
echo "-----"
```

```
# 7. Append
cat myfile >> myappendfile
echo "Content appended to myappendfile"
```

Example Output

First 2 rows:
1001 RAM 97
1002 NEHA 89

Last 2 rows:
1012 ARUN 99
1013 NISHA 76

Lines: 6
Words: 18
Characters: 90

3rd row:
1010 DIVYA 77

Explanation

- `cat >` → create file
- `head` → first lines
- `tail` → last lines

- `cp` → copy
 - `wc` → count
 - `sed -n '3p'` → print specific line
 - `>>` → append
-

Result

Successfully performed file operations like **create**, **display**, **copy**, **count**, **extract**, and **append** using shell commands.